

R syntax and output for supplemental chapter: Basic Matrix Algebra for Statistical Modeling

Matrix algebra is more easily understood if one works through some calculations. The syntax in this document reproduces the matrix calculations presented in the supplementary text; cross-check the results produced by the calculations below against the calculations presented in the supplementary text.

Simple matrix operations

Define 3 x 2 matrix **A**; 'nrow' refers to the number of rows and 'ncol' refers to the number of columns:

```
A <- matrix(data=c(1,.2,8.5,3,4.6,-2.3), nrow=3, ncol=2)
```

Print **A** to the console window:

```
A
##      [,1] [,2]
## [1,]  1.0  3.0
## [2,]  0.2  4.6
## [3,]  8.5 -2.3
```

Create **A'** and print the result to the console window:

```
At <- t(A)
At
##      [,1] [,2] [,3]
## [1,]    1  0.2  8.5
## [2,]    3  4.6 -2.3
```

Now, take the transpose of **At**, i.e., **(A')**, and see that it is the same matrix as **A**:

```
t(At)
##      [,1] [,2]
## [1,]  1.0  3.0
## [2,]  0.2  4.6
## [3,]  8.5 -2.3
A
```

```
##      [,1] [,2]
## [1,]  1.0  3.0
## [2,]  0.2  4.6
## [3,]  8.5 -2.3
```

Define 3 x 3 matrix **C**:

```
C <- matrix(data=c(-5,2,7,1,2,3,3,6,-4), nrow=3, ncol=3)
C
##      [,1] [,2] [,3]
## [1,]   -5    1    3
## [2,]    2    2    6
## [3,]    7    3   -4
```

Calculate the trace of **C**:

```
sum(diag(C))
## [1] -7
```

See that **C** is not symmetric because **C'** is not the same as **C**:

```
t(C)
##      [,1] [,2] [,3]
## [1,]   -5    2    7
## [2,]    1    2    3
## [3,]    3    6   -4
C
##      [,1] [,2] [,3]
## [1,]   -5    1    3
## [2,]    2    2    6
## [3,]    7    3   -4
```

Define 3 x 3 matrix **D**:

```
D <- matrix(data=c(-5,1,3,1,2,6,3,6,-4), nrow=3, ncol=3)
D
##      [,1] [,2] [,3]
## [1,]   -5    1    3
## [2,]    1    2    6
## [3,]    3    6   -4
```

D is symmetric because **D'** and **D** are the same:

```
t(D)
##      [,1] [,2] [,3]
## [1,]   -5    1    3
```

```
## [2,]    1    2    6
## [3,]    3    6   -4
```

D

```
##      [,1] [,2] [,3]
## [1,]   -5    1    3
## [2,]    1    2    6
## [3,]    3    6   -4
```

Matrix addition and subtraction

Define new 2 x 3 matrices **A** and **B**:

```
A <- matrix(data=c(1,4,2,5,3,6), nrow=2, ncol=3)
B <- matrix(data=c(-5,3,1,0,2,-4), nrow=2, ncol=3)
```

Calculate **C** = **A** + **B** and check the result:

```
C <- A + B
C
```

```
##      [,1] [,2] [,3]
## [1,]   -4    3    5
## [2,]    7    5    2
```

Calculate **D** = **A** - **B** and check the result:

```
D <- A - B
D
```

```
##      [,1] [,2] [,3]
## [1,]    6    1    1
## [2,]    1    5   10
```

Calculate **E** = -**B** and check the result:

```
E <- -1*(B)
E
```

```
##      [,1] [,2] [,3]
## [1,]    5   -1   -2
## [2,]   -3    0    4
```

Multiplication with matrices

Define a new 3 x 2 matrix **A**:

```
A <- matrix(data=c(1,2,8,3,5,-2), nrow=3, ncol=2)
```

Calculate $\mathbf{B} = 3\mathbf{A}$ and check the result:

```
B <- 3*(A)
B
```

```
##      [,1] [,2]
## [1,]    3    9
## [2,]    6   15
## [3,]   24   -6
```

Define new 3 x 2 matrix $\mathbf{A}$ and new 2 x 2 matrix $\mathbf{B}$:

```
A <- matrix(data=c(2,4,6,1,2,3), nrow=3, ncol=2)
B <- matrix(data=c(1,3,3,1), nrow=2, ncol=2)
```

Calculate matrix product $\mathbf{C} = \mathbf{AB}$.

It is *critical* to recognize that the matrix product operator in R is ‘%%’; a *different calculation is carried out if you simply use* ”.

```
C <- A%%B
```

Check the result:

```
C
##      [,1] [,2]
## [1,]    5    7
## [2,]   10   14
## [3,]   15   21
```

What happens when you try to multiply $\mathbf{A}$ and $\mathbf{B}$ in the opposite order to form $\mathbf{X} = \mathbf{BA}$?

```
#X <- B%*%A
```

But you can calculate $\mathbf{X} = \mathbf{BA}'$:

```
X <- B%*%t(A)
```

Check the result; how does it compare to $\mathbf{C} = \mathbf{AB}$?

```
X
##      [,1] [,2] [,3]
## [1,]    5   10   15
## [2,]    7   14   21

C
##      [,1] [,2]
## [1,]    5    7
## [2,]   10   14
## [3,]   15   21
```

Define new 2 x 2 matrices **D** and **E**:

```
D <- matrix(data=c(1,3,2,4), nrow=2, ncol=2)
E <- matrix(data=c(0,2,3,1), nrow=2, ncol=2)
```

Calculate matrix product **F = DE** and check the result:

```
F <- D%%E
F
##      [,1] [,2]
## [1,]    4    5
## [2,]    8   13
```

Notice that a different result is obtained if you pre-multiply **D** by **E**:

```
E%%D
##      [,1] [,2]
## [1,]    9   12
## [2,]    5    8
```

Determinants and matrix inversion

Define a new 3 x 3 matrix **A**:

```
A <- matrix(data=c(3,4,2,1,1,3,4,5,5), nrow=3, ncol=3)
```

Get the determinant of **A**; confirm that **A** is singular:

```
det(A)
## [1] 0
```

Define a new 2 x 2 matrix **A**:

```
A <- matrix(data=c(2,1,9,4), nrow=2, ncol=2)
```

Get the inverse of **A** using the 'solve' operator and check the result:

```
Ainv <- solve(A)
Ainv
##      [,1] [,2]
## [1,]   -4    9
## [2,]    1   -2
```

Confirm that **Ainv** is the inverse of **A**:

```
A%%Ainv
```

```
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    1
```

$A^{-1} \cdot A$

```
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    1
```

Define 2 x 1 vector **y** and new 2 x 2 matrix **A**:

```
y <- matrix(data=c(-5,7), nrow=2, ncol=1)
A <- matrix(data=c(2,1,9,4), nrow=2, ncol=2)
```

Solve for **x** as $A^{-1}y$ and check the result:

```
x <- solve(A)%*%y
x
##      [,1]
## [1,]    83
## [2,]   -19
```

Matrix calculations for statistical applications

Means

Input data as a column vector **X**:

```
X <- matrix(data=c(3,4,6,2,5), nrow=5, ncol=1)
```

Define conformable unit vector **unitV**:

```
#the number of rows of X is the sample size
N <- nrow(X)
#create a row vector of ones with length N
unitV <- rep(1,N)
```

```
#Although the code above is more generalizable, the unit vector also could
have been defined like this:
#unitV <- matrix(data=c(1,1,1,1,1), nrow=1, ncol=5)
```

Calculate the mean and check the result:

```
meanX <- (unitV%*%X)/N
meanX
##      [,1]
## [1,]    4
```

Input new data as an $N \times P$ (5 x 3) data matrix **X**:

```
X <- matrix(data=c(3,4,6,2,5,7.8,8,8.2,7.9,8.1,0,0,1,1,1), nrow=5, ncol=3)
```

Calculate the vector of means and check the result:

```
N <- nrow(X)
unitV <- rep(1,N)
meansX <- (unitV%*%X)/N
meansX

##      [,1] [,2] [,3]
## [1,]    4    8  0.6
```

Variance and covariance

Calculate vector of deviation scores from **X** and check the result:

```
Xd <- X - (unitV%*%meansX)
Xd

##      [,1] [,2] [,3]
## [1,]   -1 -0.2 -0.6
## [2,]    0  0.0 -0.6
## [3,]    2  0.2  0.4
## [4,]   -2 -0.1  0.4
## [5,]    1  0.1  0.4
```

Calculate the covariance matrix **S** and check the result:

```
S <- (t(Xd)%*%Xd)/(N-1)
S

##      [,1] [,2] [,3]
## [1,] 2.500 0.225 0.25
## [2,] 0.225 0.025 0.05
## [3,] 0.250 0.050 0.30
```

Standard deviation

Extract the variances from the covariance matrix. Rather than creating a diagonal matrix, the 'diag' operator will create a vector from the diagonal elements of **D**:

```
D <- diag(S)
```

Then calculate the standard deviations and check the result:

```
stdevs <- D^.5
stdevs

## [1] 1.5811388 0.1581139 0.5477226
```

Correlations

First, need to place the standard deviations obtained above back into a diagonal matrix, then get its inverse to form $\mathbf{D}^{-1/2}$:

```
D <- diag(stdevs)
Dinv <- solve(D)
```

Then calculate the correlation matrix and check the result:

```
R <- Dinv%%S%%Dinv
R
##           [,1]      [,2]      [,3]
## [1,] 1.0000000 0.9000000 0.2886751
## [2,] 0.9000000 1.0000000 0.5773503
## [3,] 0.2886751 0.5773503 1.0000000
```

Matrix equations for linear regression

Below, we use the data file 'generic.csv' to confirm that the matrix formulas presented in Chapter 7 for OLS multiple regression produce the same results as the regular 'lm' function used to estimate a regression model in R.

```
dat <- read.csv("generic.csv", header=T)
```

Take a look at the first 6 rows of the data:

```
head(dat)
##   const  x1  x2  x3  y
## 1     1 3.88 8.05 3.49 9.27
## 2     1 6.13 9.44 2.94 10.02
## 3     1 4.49 6.54 5.00 10.10
## 4     1 5.42 8.67 2.86  9.03
## 5     1 6.16 9.57 3.26 11.97
## 6     1 4.97 8.53 3.05 10.21
```

In this example, we would like to use the OLS matrix formula to estimate the parameters (intercept and slope coefficients) for the linear regression of 'y' on 'x1', 'x2', and 'x3'.

Notice above that a column of 1s (called 'const' for 'constant') is included for your convenience: As explained in Chapter 7, it is important for the constant to be included in **X** so that the intercept parameter can be estimated.

First, need to partition the data set to obtain a separate data matrix **X** for the predictors (including the constant) and **Y** for the outcome variable:

```
Y <- as.matrix(dat[,5]) #Y gets only the 5th column of 'dat'
X <- as.matrix(dat[, -5]) #X gets all columns of 'dat' except the 5th column
```

Calculate the regression parameter estimates and check the result:


```

Xt <- t(X)
betaHat <- solve(Xt**X)**Xt**Y
betaHat

##           [,1]
## const 3.4182221
## x1    0.2301395
## x2    0.4708032
## x3    0.5236967

```

The output above gives the estimated intercept as 3.42, estimated coefficient for x1 is 0.23, coefficient for x2 is 0.47, coefficient for x3 is 0.52.

Calculate the vector of predicted values and check the result:

```

Yhat <- X**betaHat
Yhat

##           [,1]
## [1,]  9.928831
## [2,] 10.813028
## [3,] 10.149085
## [4,] 10.245215
## [5,] 11.048720
## [6,] 10.175242
## [7,] 10.077627
## [8,] 11.245702
## [9,] 11.754630
## [10,]  8.855757
## [11,]  9.543531
## [12,]  9.386680
## [13,] 10.635065
## [14,] 11.223017
## [15,] 10.680342
## [16,] 10.318285
## [17,]  9.839223
## [18,] 10.012895
## [19,] 12.099364
## [20,] 10.887762

```

Calculate the vector of predicted values using the hat matrix, **H** and check that the result is the same as above:

```

H <- X ** solve(Xt**X)**Xt
Yhat <- H**Y
Yhat

##           [,1]
## [1,]  9.928831
## [2,] 10.813028
## [3,] 10.149085

```

```
## [4,] 10.245215
## [5,] 11.048720
## [6,] 10.175242
## [7,] 10.077627
## [8,] 11.245702
## [9,] 11.754630
## [10,] 8.855757
## [11,] 9.543531
## [12,] 9.386680
## [13,] 10.635065
## [14,] 11.223017
## [15,] 10.680342
## [16,] 10.318285
## [17,] 9.839223
## [18,] 10.012895
## [19,] 12.099364
## [20,] 10.887762
```

Extract leverage values from the hat matrix:

```
HatValues <- diag(H)
```

```
HatValues
```

```
## [1] 0.15009779 0.22562889 0.51977454 0.11348985 0.20060162 0.06459411
## [7] 0.07555697 0.38185181 0.23641295 0.31060137 0.13074025 0.16859782
## [13] 0.17120423 0.23344661 0.30485099 0.07330064 0.16351153 0.08730361
## [19] 0.30780193 0.08063250
```

Calculate the residuals and check the result; confirm that the mean of the residuals is 0 (within a very small amount of rounding error):

```
e <- Y - Yhat
```

```
e
```

```
##           [,1]
## [1,] -0.658830864
## [2,] -0.793028085
## [3,] -0.049085188
## [4,] -1.215214806
## [5,] 0.921280362
## [6,] 0.034758058
## [7,] 0.572372967
## [8,] 0.384297900
## [9,] 1.205370232
## [10,] 0.224243306
## [11,] -0.463531334
## [12,] 1.673320272
## [13,] 0.624935055
## [14,] -0.283016878
## [15,] 0.169658249
## [16,] -0.198284704
```

```
## [17,] -0.009222885
## [18,] -0.582895013
## [19,] -0.289364374
## [20,] -1.267762269

round(mean(e), digits=10)

## [1] 0
```

Next, return to the original dataset to use the 'lm' function to estimate the regression model:

```
model <- lm(y~x1+x2+x3, data=dat)
```

Confirm that the parameter estimates, predicted values, leverage values, and residuals match the results calculated above using matrix formulas:

```
model

##
## Call:
## lm(formula = y ~ x1 + x2 + x3, data = dat)
##
## Coefficients:
## (Intercept)          x1          x2          x3
##      3.4182      0.2301      0.4708      0.5237

fitted(model)

##           1           2           3           4           5           6           7
##  9.928831 10.813028 10.149085 10.245215 11.048720 10.175242 10.077627
## 11.245702
##           9          10          11          12          13          14          15
## 11.754630  8.855757  9.543531  9.386680 10.635065 11.223017 10.680342
## 10.318285
##          17          18          19          20
##  9.839223 10.012895 12.099364 10.887762

hatvalues(model)

##           1           2           3           4           5           6
##  0.15009779 0.22562889 0.51977454 0.11348985 0.20060162 0.06459411
## 0.07555697
##           8           9          10          11          12          13
##  0.38185181 0.23641295 0.31060137 0.13074025 0.16859782 0.17120423
## 0.23344661
##          15          16          17          18          19          20
##  0.30485099 0.07330064 0.16351153 0.08730361 0.30780193 0.08063250
```

```
residuals(model)

##           1           2           3           4           5
6
## -0.658830864 -0.793028085 -0.049085188 -1.215214806  0.921280362
0.034758058
##           7           8           9          10          11
12
##  0.572372967  0.384297900  1.205370232  0.224243306 -0.463531334
1.673320272
##          13          14          15          16          17
18
##  0.624935055 -0.283016878  0.169658249 -0.198284704 -0.009222885 -
0.582895013
##          19          20
## -0.289364374 -1.267762269
```

Eigenvalues and eigenvectors

We can see the eigenstructure of the correlation matrix calculated above using the ‘eigen’ function:

```
eigen(R)

## eigen() decomposition
## $values
## [1] 2.21399530 0.74132436 0.04468034
##
## $vectors
##           [,1]      [,2]      [,3]
## [1,] -0.5974811 -0.5095952  0.6191357
## [2,] -0.6597262 -0.1265119 -0.7407807
## [3,] -0.4558263  0.8510626  0.2606048
```

The following code confirms that $\mathbf{R} = \mathbf{U}\mathbf{D}\mathbf{U}'$ where $\mathbf{U}$ is a matrix formed by the three eigenvectors and $\mathbf{D}$ is a diagonal matrix of the eigenvalues:

```
eigStructure <- eigen(R)
U <- eigStructure$vectors
D <- diag(eigStructure$values)

R <- U%*%D%*%t(U)
R

##           [,1]      [,2]      [,3]
## [1,] 1.0000000 0.9000000 0.2886751
## [2,] 0.9000000 1.0000000 0.5773503
## [3,] 0.2886751 0.5773503 1.0000000
```

The matrix **R** above is the same as the correlation matrix calculated earlier.